

PRD Script Labs V2.0

Script Labs App

Dokumen requirement acuan untuk desain test case, automation, dan performance testing
QA

Product Requirements Document (PRD) - Versi 2.0

Script Labs App — API Latihan untuk QA

Informasi Dokumen

- **Versi:** 2.0 (ditulis ulang agar sesuai dengan API yang benar-benar sudah dibangun)
 - **Tanggal:** 13 September 2026
 - **Status:** Aktif / Acuan Utama
 - **Tujuan:** Dokumen ini adalah **sumber kebenaran untuk perilaku yang diharapkan (expected behavior)**. Ditulis agar seorang QA bisa langsung menurunkan test case (positif, negatif, boundary, keamanan, performa) dari sini dan menjalankannya terhadap API yang sesungguhnya.
 - **Terkait:** [PRD V1.0](#) (baseline historis), [Dokumentasi API](#), [Arsitektur Database](#)
-

1. Ringkasan Eksekutif

1.1 Gambaran Produk

Script Labs adalah REST API kecil (Node.js/Express + PostgreSQL) yang dibangun untuk memberi QA **target latihan yang realistis** — bukan produk SaaS produksi. Aplikasi ini menyediakan autentikasi user berbasis JWT dan resource "lab" (CRUD + pencarian), yang di-deploy di server sungguhan (Vultr VPS) sehingga berperilaku seperti environment nyata: ada latensi jaringan asli, rate limit asli, dan jalur error asli.

1.2 Kenapa aplikasi ini dibuat (Tujuan Bisnis)

- Memberi pembelajar QA **API yang aman dan berisiko rendah** untuk berlatih seluruh jalur skill QA secara berurutan:
 1. **Desain test case** — test case fungsional, negatif, boundary, dan keamanan yang diturunkan dari PRD ini.
 2. **Automation testing** — menulis script test API (Postman/Newman, REST-assured, Playwright API testing, dll.) terhadap backend yang benar-benar sudah di-deploy.
 3. **Performance testing** — menjalankan load/stress test (k6, JMeter, Artillery) terhadap VPS nyata dengan sumber daya terbatas, dan mengamati bottleneck yang benar-benar terjadi (rate limit, batas connection pool database, hashing password yang CPU-intensive).
- Menyediakan codebase yang cukup sederhana untuk dibaca sepenuhnya, sehingga pembelajar bisa membandingkan "yang seharusnya" (PRD ini) vs "kenyataan" (API yang berjalan) sebagai latihan pelaporan defect.

1.3 Metrik Keberhasilan

- Pembelajar QA bisa menulis test suite lengkap (happy path + negatif + edge case) untuk setiap endpoint di dokumen ini tanpa perlu membaca source code.
 - Test suite otomatis (Postman/Newman atau berbasis kode) bisa dijalankan terhadap API yang sudah di-deploy tanpa setup manual selain mendaftarkan user test.
 - Script performance test bisa diarahkan ke API yang sudah di-deploy dan menghasilkan angka latensi/error-rate yang bermakna dan bisa direproduksi.
-

2. Ruang Lingkup

2.0 Repository Terkait

Proyek Script Labs terdiri dari **dua repository terpisah**:

Bagian	Repository	Deployment
Backend (API — dijelaskan di PRD ini)	Hendrich/script-labs	Vultr VPS
Frontend (UI)	Hendrich/script-labs-app	Vercel, domain labs.hendri.me (DNS via Cloudflare)

PRD ini **hanya mencakup backend/API**. Frontend memang ada dan sudah di-deploy — bukan tidak ada — tapi requirement dan test case UI-nya berada di luar cakupan dokumen ini dan sebaiknya dikelola di repository frontend tersebut. Kalau kamu perlu latihan test case berbasis UI (form, tombol, tampilan), gunakan repo/site frontend itu sebagai acuan, bukan dokumen ini.

2.1 Termasuk dalam Ruang Lingkup

- Registrasi user, login, logout, verifikasi sesi/token (JWT, stateless).
- CRUD + pencarian untuk satu jenis resource: "**lab**" (**title**, **description**, dimiliki oleh satu user).
- Rate limiting pada endpoint autentikasi.
- Validasi input dan response error yang konsisten.

2.2 Di Luar Ruang Lingkup (secara eksplisit BELUM diimplementasikan pada API ini — jangan buat test case API dengan asumsi ini ada)

- Test case berbasis UI/tampilan — itu ranah repository frontend ([script-labs-app](#)), bukan API ini.
- Tidak ada fitur lupa password / reset password via email.
- Tidak ada autentikasi pihak ketiga (Google/Supabase/OAuth) — autentikasi murni email+password lokal.
- Tidak ada field "sort_by" / "sort_order" / kategori / rating / ISBN pada lab — sebuah lab hanya punya **title** dan **description**.
- Tidak ada operasi bulk, tidak ada endpoint admin/metrics, tidak ada endpoint statistik user.

*Catatan untuk QA: draf-draf awal dokumen ini (dan dokumen lain di repository) pernah menyebutkan Supabase, lupa password, rating/ISBN buku, dan hal lain. Itu semua adalah draf aspirasional yang **tidak pernah diimplementasikan**. Bagian ini sengaja ditulis supaya QA tidak membuat test case untuk fitur yang tidak ada di API sesungguhnya.*

3. Kebutuhan Fungsional (Functional Requirements)

Setiap requirement di bawah menyertakan aturan level-field yang dibutuhkan untuk merancang equivalence class dan boundary value.

3.1 Autentikasi

ID	Requirement	Prioritas
AUTH-001	Pengunjung dapat mendaftar dengan email unik dan password.	Tinggi
AUTH-002	User terdaftar dapat login dengan email + password dan menerima JWT.	Tinggi

ID	Requirement	Prioritas
AUTH-003	Setiap endpoint terproteksi harus menolak request tanpa header <code>Authorization: Bearer <token></code> yang valid.	Tinggi
AUTH-004	Password tidak boleh pernah disimpan atau dikembalikan dalam bentuk plaintext.	Tinggi
AUTH-005	JWT harus punya masa kedaluwarsa (default: 24 jam) dan ditolak setelah kedaluwarsa.	Sedang
AUTH-006	Client dapat memanggil endpoint "logout"; karena auth bersifat stateless, endpoint ini cukup menandakan sukses — client bertanggung jawab menghapus token-nya sendiri.	Rendah
AUTH-007	User yang sudah login dapat mengambil profilnya sendiri (<code>/me</code>) dan memverifikasi token miliknya sendiri (<code>/verify-token</code>).	Sedang
AUTH-008	Percobaan login/registrasi yang berulang gagal dari IP yang sama harus dibatasi (throttle).	Tinggi

3.1.1 Aturan Field

Field	Aturan
email	Wajib. Harus format email yang valid secara sintaksis. Pencocokan tidak case-sensitive (mis. <code>A@B.com</code> dan <code>a@b.com</code> dianggap akun yang sama).
password	Wajib. Minimal 6 karakter, maksimal 128 karakter. Tidak ada syarat kompleksitas (tidak wajib huruf besar/angka/symbol).

3.1.2 Kebutuhan per Endpoint

POST `/api/auth/register`

- **Diberikan** email yang valid dan belum terdaftar, serta password 6-128 karakter

Ketika client mengirim registrasi

Maka API mengembalikan **201 Created** berisi JWT dan data user yang dibuat: `id`, `email`, `role` (default "user"), `status` (default "active"). Response tidak boleh pernah menyertakan password atau hash-nya.

- **Diberikan** email yang sudah terdaftar (perbandingan tidak case-sensitive)

Ketika client mengirim registrasi

Maka API mengembalikan **409 Conflict** dengan kode error `EMAIL_EXISTS`, dan tidak ada akun baru yang dibuat.

- **Diberikan** email yang kosong/tidak valid, atau password kurang dari 6 atau lebih dari 128 karakter

Ketika client mengirim registrasi

Maka API mengembalikan **400 Bad Request** yang menjelaskan field mana yang gagal validasi, dan tidak ada akun yang dibuat.

- **Diberikan** lebih dari 5 percobaan registrasi dari IP yang sama dalam 15 menit

Ketika client mengirim registrasi lagi

Maka API mengembalikan **429 Too Many Requests**.

POST /api/auth/login

- **Diberikan** email + password yang benar untuk akun yang aktif

Ketika client login

Maka API mengembalikan **200 OK** berisi JWT dan data user: id, email, role, status.

- **Diberikan** email yang tidak terdaftar, ATAU email benar dengan password salah

Ketika client login

Maka API mengembalikan **401 Unauthorized** dengan pesan generik yang **sama persis** untuk kedua kasus (mis. "Invalid email or password") — API tidak boleh pernah membocorkan apakah email tersebut terdaftar (anti user-enumeration).

- **Diberikan** email + password benar untuk akun yang status-nya "locked"

Ketika client login

Maka API mengembalikan **403 Forbidden**, bukan 401.

- **Diberikan** email atau password yang kosong/tidak valid

Ketika client login

Maka API mengembalikan **400 Bad Request**.

- **Diberikan** lebih dari 5 percobaan login dari IP yang sama dalam 15 menit

Ketika client login lagi

Maka API mengembalikan **429 Too Many Requests**, terlepas dari benar-tidaknya kredensial pada percobaan ke-6 tersebut.

POST /api/auth/logout

- **Diberikan** request apa pun, dengan atau tanpa token

Ketika client memanggil logout

Maka API mengembalikan **200 OK** yang mengonfirmasi logout. (JWT stateless — server tidak menyimpan sesi untuk di-invalidate; endpoint ini murni sebagai konfirmasi sisi client.)

GET /api/auth/me

- **Diberikan** token yang valid dan belum kedaluwarsa

Ketika client meminta profilnya

Maka API mengembalikan **200 OK** berisi id, email, role, status, created_at user tersebut.

- **Diberikan** token yang kosong, tidak valid, atau sudah kedaluwarsa

Ketika client meminta profilnya

Maka API mengembalikan **401 Unauthorized**.

POST /api/auth/verify-token

- **Diberikan** token yang valid dan belum kedaluwarsa

Ketika client memverifikasinya

Maka API mengembalikan **200 OK** dengan valid: true, user_id, email, dan waktu kedaluwarsa token.

- **Diberikan** token yang kosong, tidak valid, atau sudah kedaluwarsa

Ketika client memverifikasinya

Maka API mengembalikan **401 Unauthorized**.

3.2 Manajemen Lab (CRUD)

Semua endpoint di bawah `/api/labs` membutuhkan header `Authorization: Bearer <token>` yang valid. Seorang user hanya bisa melihat dan mengubah lab **miliknya sendiri**.

ID	Requirement	Prioritas
LAB-001	User yang login dapat melihat daftar lab miliknya, dengan pagination.	Tinggi
LAB-002	User yang login dapat mengambil satu lab miliknya berdasarkan ID.	Tinggi
LAB-003	User yang login dapat membuat lab baru dengan <code>title</code> dan <code>description</code> .	Tinggi
LAB-004	User yang login dapat mengubah <code>title</code> dan/atau <code>description</code> dari lab miliknya.	Tinggi
LAB-005	User yang login dapat menghapus lab miliknya.	Tinggi
LAB-006	User yang login dapat mencari lab miliknya berdasarkan kata kunci pada <code>title</code> atau <code>description</code> , dengan pagination.	Sedang
LAB-007	Seorang user tidak boleh pernah bisa melihat, mengubah, atau menghapus lab milik user lain, sekalipun dengan menebak ID numeriknya.	Tinggi
LAB-008	Membuat lab dengan <code>title</code> dan <code>description</code> yang identik dengan lab yang sudah ada (untuk user yang sama) harus ditolak sebagai duplikat.	Sedang

3.2.1 Aturan Field

Field	Aturan
<code>title</code>	Wajib saat membuat lab. 1-255 karakter (setelah di-trim).
<code>description</code>	Wajib saat membuat lab. 1-1000 karakter (setelah di-trim).
<code>id</code> (path param)	Harus berupa bilangan bulat positif.
<code>page</code> (query param)	Bilangan bulat positif, default <code>1</code> .
<code>limit</code> (query param)	Bilangan bulat positif, default <code>10</code> , maksimum <code>100</code> .

3.2.2 Kebutuhan per Endpoint

GET `/api/labs`

- **Diberikan** user yang login dengan N lab

Ketika memanggil endpoint ini tanpa query param

Maka API mengembalikan **200 OK** berisi lab miliknya (default 10 per halaman), diurutkan dari yang terbaru, plus metadata pagination (`page`, `limit`, `total`, `totalPages`).

- **Diberikan** query param opsional `search`

Ketika diisi

Maka hasil difilter ke lab yang `title` atau `description`-nya mengandung teks pencarian (tidak case-sensitive).

- **Diberikan** nilai `page` atau `limit` yang bukan bilangan bulat positif yang valid (mis. teks, negatif, nol, desimal)

Ketika client memanggil endpoint ini

Maka API harus mengembalikan **400 Bad Request** dengan pesan validasi yang jelas — tidak boleh 500, dan tidak boleh membocorkan pesan error database mentah.

GET `/api/labs/search`

- Kontrak sama dengan `GET /api/labs`, tapi query param bernama `q`, bukan `search`, dan response menyertakan echo dari kata kunci pencarian (`search_query`).
- **Diberikan** `q` yang kosong atau tidak diisi

Ketika client mencari

Maka API mengembalikan semua lab milik user (dengan pagination), setara dengan `GET /api/labs`.

- Aturan validasi sama seperti di atas: `page/limit` yang tidak valid harus mengembalikan **400**, tidak boleh 500 atau error database mentah.

GET `/api/labs/:id`

- **Diberikan** `id` yang ada dan milik pemanggil

Ketika client memintanya

Maka API mengembalikan **200 OK** berisi lab tersebut.

- **Diberikan** `id` yang tidak ada, atau ada tapi milik user lain

Ketika client memintanya

Maka API mengembalikan **404 Not Found** — response harus identik untuk kedua kasus (jangan bocorkan apakah ID tersebut ada tapi milik orang lain).

- **Diberikan** `id` yang bukan bilangan bulat positif (mis. `abc`, `-1`, `1.5`)

Ketika client memintanya

Maka API mengembalikan **400 Bad Request**.

POST `/api/labs`

- **Diberikan** `title` (1-255 karakter) dan `description` (1-1000 karakter) yang valid

Ketika client membuat lab

Maka API mengembalikan **201 Created** berisi lab baru, lengkap dengan `id`, `created_at`, `updated_at` yang dihasilkan.

- **Diberikan** pasangan `title+description` yang identik dengan lab yang sudah dimiliki user yang sama

Ketika client mencoba membuatnya lagi

Maka API mengembalikan **409 Conflict**.

- **Diberikan** `title` kosong, `description` kosong, atau salah satunya di luar batas panjang (kosong setelah trim, atau melebihi maksimum)

Ketika client membuat lab

Maka API mengembalikan **400 Bad Request** yang menjelaskan field mana yang tidak valid.

PUT /api/labs/:id

- Diberikan id lab yang dimiliki dan ada, plus minimal satu dari title/description di body

Ketika client mengubahnya

Maka API mengembalikan **200 OK** berisi lab yang sudah diperbarui, dan updated_at berubah.

- Diberikan body kosong (tidak ada field untuk diubah)

Ketika client mencoba mengubah

Maka API mengembalikan **400 Bad Request**.

- Diberikan id yang tidak ada atau bukan milik pemanggil

Ketika client mencoba mengubah

Maka API mengembalikan **404 Not Found**.

DELETE /api/labs/:id

- Diberikan id lab yang dimiliki dan ada

Ketika client menghapusnya

Maka API mengembalikan **200 OK** yang mengonfirmasi penghapusan, dan GET berikutnya pada id tersebut mengembalikan 404.

- Diberikan id yang tidak ada atau bukan milik pemanggil

Ketika client mencoba menghapusnya

Maka API mengembalikan **404 Not Found**.

4. Kebutuhan Non-Fungsional

4.1 Keamanan

ID	Requirement
SEC-001	Password harus di-hash (tidak boleh pernah disimpan atau di-log dalam bentuk plaintext).
SEC-002	Semua request yang mengubah state (POST/PUT/DELETE) harus ditolak jika header Origin/Referer-nya tidak cocok dengan daftar origin yang diizinkan.
SEC-003	Endpoint autentikasi harus membatasi (rate-limit) berdasarkan IP untuk menahan brute-force dan credential-stuffing.
SEC-004	Response error di production tidak boleh pernah membocorkan stack trace, pesan error database mentah, atau path file internal.
SEC-005	Seorang user tidak boleh pernah bisa mengakses atau mengubah data user lain lewat endpoint apa pun.

4.2 Performa

ID	Requirement	Target
PERF-001	Waktu respons GET /api/labs / GET /api/labs/search dalam kondisi normal	< 500 ms (p95)
PERF-002	Waktu respons POST /api/auth/login (bcrypt sengaja CPU-intensive; endpoint ini diperkirakan paling lambat)	< 1000 ms (p95)
PERF-003	API harus degradasi secara graceful (response error yang jelas, bukan hang atau crash) ketika rate limit atau connection pool database yang dikonfigurasi terlampaui — inilah yang seharusnya diuji oleh performance/load testing pada aplikasi ini.	

4.3 Reliabilitas & Kontrak Error

ID	Requirement
REL-001	Setiap response error harus menyertakan message yang bisa dibaca manusia.
REL-002	Error validasi harus mengembalikan 400, error autentikasi 401/403, tidak-ditemukan 404, konflik 409, rate-limit 429, dan kegagalan server tak terduga 500 — status code harus digunakan secara konsisten di semua endpoint.
REL-003	Input yang salah bentuk atau tak terduga tidak boleh pernah membuat server crash atau mengembalikan 500 yang tidak tertangani padahal seharusnya 400 — ini aturan paling penting untuk desain test case negatif terhadap API ini.

5. Panduan Desain Test Case untuk QA

PRD ini sengaja ditulis agar setiap requirement per endpoint di atas bisa langsung dipetakan menjadi Given/When/Then dan menjadi satu atau lebih test case. Cakupan yang disarankan per endpoint:

1. **Happy path** — input valid, verifikasi status code + bentuk response.
2. **Boundary value** — untuk setiap aturan panjang/angka di atas (mis. password persis 6, 5, 128, 129 karakter; page = 0, 1, negatif, bukan angka).
3. **Negatif/keamanan** — kredensial salah, akun terkunci, mengakses resource user lain lewat ID, token kosong/kedaluwarsa/tidak valid, body JSON yang salah bentuk.
4. **Rate limiting** — melebihi batas 5 request/15 menit pada /register dan /login.
5. **Konsistensi kontrak** — bandingkan bentuk response/status code aktual dengan yang ditentukan di dokumen ini; ketidaksesuaian apa pun adalah defect yang harus dilaporkan.

Struktur ini juga yang sebaiknya dipakai untuk mengorganisir automation suite (koleksi Postman/Newman, atau test API berbasis kode), dan menjadi acuan performance test script untuk skenario beban yang realistis (mis. traffic berkelanjutan pada GET /api/labs, traffic burst pada POST /api/auth/login untuk mengamati perilaku rate-limiting).

6. Kriteria Keberhasilan

- [x] Registrasi, login, logout, `/me`, `/verify-token` sudah diimplementasikan dan sesuai Bagian 3.1.
- [x] CRUD + pencarian lab sudah diimplementasikan dan sesuai Bagian 3.2.
- [x] Rate limiting aktif pada `/register` dan `/login`.
- [] Semua requirement di Bagian 3 sudah diverifikasi oleh automated test suite (deliverable QA).
- [] Baseline performa untuk target di Bagian 4.2 sudah ditetapkan (deliverable QA).

Status Dokumen: Aktif / Acuan Utama

Terakhir Diperbarui: 13 September 2026

Pemilik: Hendri Christianto